Certified Rewrite Discovery for Tensor Programs: A Machine-Minted Conv+BN Fold from Minting to Runtime

Draft — proposed, not accepted

mlopt project

2026-10-20

Abstract

Compilers choose better implementations only from transformations a human already wrote. We describe a pipeline in which the rewrite rules themselves are a product of measurement: laws are data, composites are minted by a fixpoint over the shipped library, guards are transported through an evidence store, derivability is credited against an object's own premise closure, and admission is decided by a staged gauntlet that has repeatedly falsified the engine's own candidates. The flagship result is `conv_bn_fold` — the eval-mode batch-norm weight fold a hand optimizer writes — minted, admitted, and shipped verbatim as a machine-authored law in the default rule set. The deployed program evaluates bitwise-equal to the spelled one through an independent runtime and is 16% cheaper per call (72.8 vs 87.0 ms), with the term-level cost model agreeing in direction. All reported numbers are reproducible from the committed tree. The scope is deliberately honest: a kernel-scale, certified-rewrite result, not a model-scale speedup claim.

> **!** Important
>
> **Status.** This is a working draft — *proposed, not accepted*. It has not had human revision or external review. Every quantitative claim names the probe, test, or retro that produced it; the rest of the paper should be read as a candidate for rewriting, not as a settled result.

1 Introduction

A tensor-program optimizer faces four questions that are usually conflated: *optimization* (choosing a better implementation among known transformations), *discovery* (finding transformations nobody wrote), *verification* (establishing validity before acceptance), and *search* (exploring the space efficiently). Equality saturation (Tate et al. 2009; Willsey et al. 2021) answers the first and third by closing the program space under a fixed law bank and extracting a cheapest member; systems such as TENSAT apply it to tensor graphs with a hand-written rule set. Rule-inference work (Ruler (Nandi et al. 2021), the Babble line (Cao et al. 2023)) attacks discovery for scalar and library-abstraction domains, but the synthesis story for *tensor programs with runtime-priced deployment* is thinner.

This paper reports the machinery — and one complete measured traversal of it — in the mlopt engine. The claim is narrow and checkable:

> A rewrite a hand optimizer would write — the eval-mode conv+BN weight fold — was **discovered, proven, priced, and shipped by the machine**, and its deployed form runs bitwise-equal and cheaper through an independent runtime.

No model-scale speedup is claimed. The result is kernel-scale (one op pair, fp64 fixtures), the runtime is a tree-walk interpreter (debug build), and the move alphabet is bounded by an 18-op host-tensor surface — all named in Section 6. What the case study establishes is the *pipeline*: that a law can travel from measured need to a shipped, certified, runtime-priced rule without a human writing it.

The repository is `handl-lang/research-mlopt`; every number below cites the artifact that produced it (§Section 7).

2 Background

2.1 Equality saturation

Given a term and a set of rewrite laws, an e-graph compactly represents the closure: all programs reachable by the laws, sharing subterms, until a fixed point (Tate et al. 2009). Extraction then picks a cheapest member under a cost model. The soundness boundary is the law bank: any unsound rule poisons every e-class it touches, so in mlopt a rule's side conditions (`check`/`derive`/declarative `cond`) are the *only* layer that keeps a false member out — the certificate replay re-proves that a step is a valid application, not that the rule is true.

2.2 Laws as data on a categorical substrate

Catopt reads a program as a 1-morphism between tensor types, a rewrite law as a 2-cell between programs, and a derivation — one law proven from premises — as a 3-cell. The consequence that matters operationally: laws are **data rows** (an LHS/RHS pattern pair plus a guard spec), serializable, storable, composable. The shipped bank (`mlopt_core.laws.ALL_RULES`, 84 rules) mixes human-authored axioms with twelve machine-admitted objects (`mlopt_core.laws.provenance.MACHINE_SHIPPED`).

2.3 The handl substrate and mlopt-kit

The same "one language, many interpretations" model is implemented by the handl platform: eval, typecheck and rewrite are interpretations of one language whose tables are data. `packages/mlopt-kit` — a plain Rust crate — composes handl's canonical `Language`, its lex/parse/tree-walk-exec engines, and `handl-host-tensor`'s 18-op `tensor.*` surface as dependencies (no forks). `hnl-run` is a subprocess door: `.hnl` in, a JSON wire tensor out, `--repeat N` evaluating a source N times in one process — the measurement hook. `lab/kit/term_hnl.py` renders any mlopt IR term as a `.hnl` program from a data table (per-op arg assembly, attr marshaling, leaf binding), so the *same object the e-graph rewrites* is what the runtime executes.

3 Machinery

3.1 The mint fixpoint

`mlopt_core.meta.synthesize_rules` runs a bounded fixpoint over the law bank on a seed term: each round applies rules, composes the pairs that chain, and emits `syn_*` composites with provenance — the names of the premise rules each composite was built from. On the eval-mode ConvBN seed it emits 974 rules in 12 s (fuel 30,000, 4 rounds); the fold composite mints deterministically (identical names across runs).

3.2 Guard transport

A minted rule's side conditions are closures over its premise objects, so the evidence store keeps them as *specs*, not code: each stored record carries a `"guard"` field — parent names plus re-expression maps — and `evidence.admit_object` rebuilds `check`/`derive` by resolving parents as store rows, transitively. The shipped form (`mlopt_core.laws.minted`) embeds the stored-closure records as data and replays the same rebuild semantics at import — louder than the store: a parent resolving short of declared strength raises rather than admitting a degraded rule.

3.3 Premise-anchored derivability

`verify_law` answers "is this instance derivable under a rule bank?" with a replayable witness. For a stored composite the honest question is *"derivable under the premises it was minted from"* — so the derivability leg evaluates against `base_rules` plus the object's transitive premise closure, with an anti-gaming bar: the certificate must be standalone-replayable and every step naming a non-axiom rule must *site-verify* under the fp64 numeric oracle. A premise false at its applied site revokes the credit and the verdict falls back to the axioms-only answer.

3.4 One cost alphabet

The cost model credits compile-time work structurally: a subtree closed over `Param`/`Const` leaves whose root op is in `FOLD_OPS` — the 161-op fold-at-lowering alphabet, single-sourced between the lowerer, the cost predicate and the extractor — is priced at ~0 marginal. This is what lets a fold's pay leg measure what it should: the fold's `mul`/`reshape`/`rsqrt` chain is once-per-deployment work, not per-call work, and `dag_cost` prices it that way.

3.5 The admission gauntlet

`evidence.run_gauntlet` stages the verdict: reconstruct → full-data → measure → truth → novelty → typed-pay → closure → certificate. Truth is two legs — the numeric oracle over the guarded region *and* premise-anchored derivability; novelty refuses re-spellings of shipped laws (`relation=duplicate`); typed-pay suppresses cost drops that mint ill-typed members; closure bounds the firing footprint. The gauntlet has repeatedly falsified the engine's own candidates — a naive broadcast guard with 212 bad accepts, an apparent paying sibling with semantically unequal sites, four deliberate false twins (`project/retros/mul-unsq-nat.md`, `layer3-loop.md`, `typed-pay-gate.md`).

3.6 Learned guidance as a game

Because laws are data, discovery admits a well-defined game structure: rule applications and compositions are well-formed moves; the admission gauntlet is the referee — a move only scores if its artifact survives admission; the cost alphabet is the score. This is implemented machinery, not aspiration: `engine` defines the `Board` protocol and episode/train/eval drivers, `players` a `LinearPolicy` arm over a domain-specified featurizer, and `play` a domain registry (`meta`, `joint`, `search`, `torch`, `gen`, `open`; `torch` and `gen` ship `deliver` hooks that lower the winning extraction to a verified module). A domain is data — a case generator, a board factory, and a feature schema — not a subclass.

Two linear priors ship with pinned held-out wins: the open-game policy ranks law candidates at ~3× the cost-delta heuristic's average precision (AP 0.133 vs 0.045) and ~2× useful laws per episode, and the meta-arena prior beats the scripted playbook (mean episode reward −0.06 vs −0.10) — the latter flagged by its own eval as possibly sampling-variance-riding, an honest negative kept in the record. A

distilled contraction policy (`PairPolicyNet`) beats `opt_einsum`'s randomized greedy (0.91× cost at n=40) and ships as a standalone artifact. Conversely, a neural rule-value policy *loses* to the heuristic (AP 0.023), and two learned-policy axes (schedule guidance; direct law invention) were measured and falsified — the evidence for learned guidance is real but early: linear and distilled arms beat baselines; neural methods do not yet.

4 Case study: the conv+BN fold

The rewrite is the standard eval-mode fold:

$$\mathrm{BN}_{\gamma,\beta,\mu,\sigma^2,\varepsilon}(\mathrm{conv}(x,W,b)) \;=\; \mathrm{conv}(x,\; W \odot \hat{s}',\; b \odot s + (\beta - s \odot \mu)), \qquad s = \gamma \cdot (\sigma^2+\varepsilon)^{-1/2}.$$

The account below is the probe's own output (`lab/probes/conv_fold_premise_pay_probe.py`, ~13 s).

Anchor. `CONV_FOLD_RULES` provides the channel-scale and channel-shift anchors (`mul(conv2d(x,W,b),ŝ)`-shaped single-step folds); `BATCH_NORM_RULES` expands `batch_norm` into its affine spelling. Individually they are enabled-but-opt-in — the anchors refuse at truth-starvation, the expand at typed-pay — their value is *enablement*: they are the fold's derivation premises.

Mint. `synthesize_rules` over the bank minus the already-shipped composite emits 974 rules; the fold composite — `batch_norm(conv2d(·),…)` → `conv2d(·)` — mints as a depth-4 `syn_*` chain over an 8-rule transitive premise closure (five shipped anchors + three intermediates).

Transport and truth. Stored with its closure, `admit_object` rebuilds both hooks from guard records (`missing=('check','derive')` → transported). The transported guard accepts the real site (1 eq / 0 ne) and 105/105 of the guarded synth region with 255 honest declines; `verify_law` replays the derivation under shipped axioms in one step — `witness=('conv_bn_fold',)`, the library law proving its own minted origin.

Pay. On the 8×8 probe site the folded instance drops `dag_cost` **34,874.3 → 17,438.7** (−50 %), fires once, verifies `pass` on the lowered modules.

Ship. The composite is promoted *verbatim*: its stored records embed in `mlopt_core.laws.minted`, its guard re-materializes at import, `MACHINE_SHIPPED` grows to 12. Re-running the gauntlet post-ship refuses at novelty with `relation=duplicate` — the documented signature of "already admitted."

Runtime-proven. Through the kit bridge (Section 2), the spelled term `batch_norm(conv2d(x,W,b),…)` and the deployed term `conv2d(x,W2,b2)` — with `W2`,`b2` themselves mlopt subterms materialized through the kit — evaluate to the same tensor (**bitwise-equal** on the seeded fixture; the full `torch.export` IR of the module evaluates at 6.7e-16 max abs diff), and per-call wall over `hnl-run --repeat 30` measures three regimes — the honest deployed arm (fold math amortized to load) is ~−5 %; an inline-fold arm that recomputes the fold per call runs *slower* (+22–33 %); and a subprocess-arm −16 % figure conflates the prelude's unused fold-marshal work into the baseline. `dag_cost` is the stable headline: 55,296 → 36,864 (−33 %). Full reconciliation: `bench/RESULTS.md` + `lab/probes/conv_fold_wall_regimes_probe.py`.

: The fold decision, priced by the runtime itself — honest regime-split. {#tbl-decision}

One honest subtlety: `dag_cost`'s param-only discount models compile-time folding the tree-walk runtime does not perform, so the measured arm separates fold-once (materialize `W2`,`b2`) from run-every-call (the deployed conv). The cost model and the wall agree in direction; the wall numbers are debug-build timings — directional, not a benchmark rig.

5 Related work

Equality saturation and e-graph superoptimization. Tate et al. (Tate et al. 2009) introduced the technique; egg (Willsey et al. 2021) made it practical; TENSAT (Yang et al. 2021) and follow-ons apply it to tensor graphs with fixed, hand-written rule sets. Catopt uses the same substrate but the rule bank is not fixed — the discovery loop writes into it through the gauntlet. egglog (Zhang et al. 2023) serves as a test-only differential oracle for the hand-rolled saturation.

Rewrite-rule inference and library learning. Ruler (Nandi et al. 2021) infers rewrite rules for scalar domains by enumeration + equality-saturation filtering — the direct ancestor of the mint-then-verify shape here, differing in that mlopt mints *composites over tensor laws* with transported guards and credits derivability against an object's own premise closure. Babble (Cao et al. 2023) learns library abstractions by anti-unification; mlopt's definitional-atom layer is the same move and measured **null** on reach-expansion — the paying layer is above it.

Compilers. TVM (Chen et al. 2018) autotunes schedules; TorchInductor (Ansel et al. 2024) fuses pointwise ops and codegen's the graph a human optimizer could reach. Both are incumbents in the repo's benchmark tables (wins and losses alike); the contribution here is not a faster kernel but a certified, machine-discovered *rewrite*.

Certified/verified compilation. The per-artifact boundary design — law, search, delivery, bound each with its own verifier — is in the spirit of translation validation; what is unusual is that the verifier also audits the *discovery* machinery (adversarial sweeps falsifying minted guards).

6 Limitations

- **Kernel scale.** One op pair, fp64 fixtures, a 1×4×24×24 activation. Nothing here measures model-scale compilation; wider-corpus evidence in the repo (bimodal exposure, ~1.5% yield of fresh paying structure) is honest that this is not yet a general speedup.
- **Move alphabet bound.** The kit surface is 18 `tensor.*` ops; the fold's own `s = ·rsqrt(²+)` is outside it (no elementwise `rsqrt`) and rides a marshaling channel. Emitted terms are confined to what the surface spells; outside-surface ops refuse by name.
- **Tree-walk runtime.** `hnl-run --repeat` timings are wall-clock on a debug-build tree-walk interpreter — they establish direction (the fold's value lands in the deployed program), not competitive throughput. No codegen claim is made.
- **Single-case gauntlet.** The admission measured here is a one-term corpus; the machinery's broader soundness evidence lives in the library-level sweeps, which caught real holes — but on different objects than this one.
- **Attr-narrow LHS.** The shipped composite's LHS carries the seed's literal attrs (`stride=(1,1)`, `padding=(1,1)`, `eps=1e-05`, `training=False`): its region is the default-attr eval-mode fold — honest, narrow, the common spelling.

7 Reproduction

All numbers are reproducible from the committed tree at `handl-lang/research-mlopt`:

```
uv sync
.venv/bin/python lab/probes/conv_fold_premise_pay_probe.py   # the admission account (~13 s)
.venv/bin/python -m pytest tests/test_kit_bridge.py -q       # the runtime-priced decision (8 t
cargo test -p mlopt-kit                                     # the Rust consumer (21 tests)
.venv/bin/python -m pytest -m "not slow"                     # the dev lane
```

The evidence ledger is `bench/RESULTS.md`; the measured retros each claim cites are under `project/retros/` (notably `minted-composite-default.md`, `kit-term-bridge.md`, `premise-anchored-derivabi fold-cost-single-source.md`); the extended working draft with the full win/loss tables is `docs/paper_draft.md`.

References

Ansel, Jason, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, et al. 2024. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation." In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 929–47. ACM. https://doi.org/10.1145/3620665.3640366.

Cao, David, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. "Babble: Learning Better Abstractions with e-Graphs and Anti-Unification." *Proceedings of the ACM on Programming Languages* 7 (POPL): 396–424. https://doi.org/10.1145/3571207.

Chen, Tianqi, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, et al. 2018. "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning." In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 578–94. USENIX Association.

Nandi, Chandrakana, Max Willsey, Amy Zhu, Yisu Remy Wang, Brett Saiki, Adam Anderson, Adriana Schulz, Dan Grossman, and Zachary Tatlock. 2021. "Rewrite Rule Inference Using Equality Saturation." *Proceedings of the ACM on Programming Languages* 5 (OOPSLA): 1–28. https://doi.org/10.1145/3485496.

Tate, Ross, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. "Equality Saturation: A New Approach to Optimization." In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 264–76. ACM. https://doi.org/10.1145/1480881.1480915.

Willsey, Max, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. "Egg: Fast and Extensible Equality Saturation." *Proceedings of the ACM on Programming Languages* 5 (POPL): 1–29. https://doi.org/10.1145/3434304.

Yang, Yichen, Yisu Remy Wang, Max Willsey, Pavel Panchekha, Zachary Tatlock, et al. 2021. "Equality Saturation for Tensor Graph Superoptimization." In *Proceedings of Machine Learning and Systems (MLSys)*, 3:255–68.

Zhang, Yihong, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2023. "Better Together: Unifying Datalog and Equality Saturation." *Proceedings of the ACM on Programming Languages* 7 (PLDI): 468–92. https://doi.org/10.1145/3591239.